

PYTHON PROGRAMMING LAB - UNIT IV

Experiment 18: Sort Words from a File

Aim:

To write a Python program that reads words from a source text file, converts all words to lowercase, sorts them alphabetically, and writes the sorted words into a new destination file.

Algorithm:

1. **Start** the program.
2. Create a source file named **source.txt** containing a mix of upper and lower-case words.
3. Open the source file in read mode ('r').
4. Read the file's content and use the **split()** method to create a list of words.
5. Convert each word in the list to **lowercase**.
6. **Sort** the list of lowercase words alphabetically using the **sort()** method.
7. Open a destination file named **destination.txt** in write mode ('w').
8. Write each word from the sorted list into the destination file, one word per line.
9. Close both files.
10. Print a success message to the console.
11. **Stop** the program.

Program:

```
18. Write a program to sort words in a file and put them in another file.  
# The output file should have only lower-case words.  
# Step 1: Create a dummy source file for the program to read from.
```

```
with open("source.txt", "w") as f:  
    f.write("This is a Sample File for testing the Python program.\n")  
    f.write("It contains Words in both Upper and lower case.")  
  
print("--- File Word Sorter ---")  
try:  
    # Open the source file for reading  
    with open("source.txt", "r") as source_file:  
        content = source_file.read()  
        words = content.split()  
  
    # Convert all words to lowercase  
    lower_case_words = [word.lower() for word in words]  
    # Sort the list of words alphabetically  
    lower_case_words.sort()
```

```
# Open the destination file for writing
with open("destination.txt", "w") as dest_file:
    for word in lower_case_words:
        dest_file.write(word + "\n")

print("Successfully sorted words from 'source.txt' and wrote to 'destination.txt'.")
except FileNotFoundError:
    print("Error: 'source.txt' not found. Please ensure the file exists.")
print("-" * 25)
```

Output:

```
--- File Word Sorter ---
Successfully sorted words from 'source.txt' and wrote to 'destination.txt'.
```

```
-----
# Content of destination.txt:
```

```
# a
# and
# both
# case
# contains
# file
# for
# in
# is
# it
# lower
# program
# python
# sample
# testing
# the
# this
# upper
# words
```

Result:

The Python program was successfully written and executed to read words from a source file, convert them to lowercase, sort them alphabetically, and write the sorted list into a new destination file (destination.txt), thereby achieving the aim of the experiment

Experiment 19: Print File Lines in Reverse

Aim:

To write a Python program that reads all lines from a text file and prints them to the console in reverse order.

Algorithm:

1. **Start** the program.
2. Create a sample file **data.txt** with multiple lines of text.
3. Open data.txt in read mode ('r').
4. Use **readlines()** to read all lines into a list.
5. **Reverse** the list of lines using slicing **[::-1]**.
6. Iterate through the reversed list and print each line, removing any trailing whitespace using **strip()**.
7. Close the file.
8. **Stop** the program.

Program:

```
# 19. Python program to print each line of a file in reverse order.
```

```
# Step 1: Create a sample file to read from
```

```
with open("data.txt", "w") as f:  
    f.write("Line 1: First line of the file.\n")  
    f.write("Line 2: This is the second line.\n")  
    f.write("Line 3: Penultimate line here.\n")  
    f.write("Line 4: The final line.\n")
```

```
print("--- Reverse File Line Printer ---")
```

```
print("Original content of 'data.txt':")
```

```
with open("data.txt", "r") as f:  
    print(f.read())
```

```
print("-" * 30)
```

```
try:
```

```
    with open("data.txt", "r") as file:  
        lines = file.readlines()  
        reversed_lines = lines[::-1]
```

```
    print("File content in reverse order:")
```

```
    for line in reversed_lines:  
        print(line.strip())
```

```
except FileNotFoundError:
```

```
print("Error: 'data.txt' not found.")  
print("-" * 30)
```

Output:

```
--- Reverse File Line Printer ---  
Original content of 'data.txt':  
Line 1: First line of the file.  
Line 2: This is the second line.  
Line 3: Penultimate line here.  
Line 4: The final line.
```

```
-----  
File content in reverse order:  
Line 4: The final line.  
Line 3: Penultimate line here.  
Line 2: This is the second line.  
Line 1: First line of the file.  
-----
```

Result:

The Python program successfully read the contents of data.txt and printed the lines to the console in reverse order, demonstrating the use of file handling and list slicing for reversing sequence elements.

Experiment 20: Count Characters, Words, and Lines

Aim:

To write a Python program that computes and displays the total number of characters, words, and lines in a text file.

Algorithm:

1. **Start** the program.
2. Create a file **report.txt** with sample text.
3. Initialize `line_count`, `word_count`, and `char_count` to **0**.
4. Open `report.txt` in read mode.
5. For each line in the file:
 - a. Increment `line_count`.
 - b. Add the length of the line to `char_count`.
 - c. Split the line into words and add the count of words to `word_count`.
6. Close the file.
7. Print the final counts.
8. **Stop** the program.

Program:

```
# 20. Python program to compute the number of characters, words and lines in a file.
```

```
# Step 1: Create a file to be analyzed
```

```
with open("report.txt", "w") as f:
```

```
    f.write("Python is a versatile programming language.\n")
```

```
    f.write("It is widely used for web development, data science, and more.\n")
```

```
print("--- File Statistics Counter ---")
```

```
try:
```

```
    line_count = 0
```

```
    word_count = 0
```

```
    char_count = 0
```

```
with open("report.txt", "r") as file:
```

```
    for line in file:
```

```
        line_count += 1
```

```
        char_count += len(line)
```

```
        words = line.split()
```

```
        word_count += len(words)
```

```
print(f"Analysis of 'report.txt':")
```

```
print(f"Number of lines: {line_count}")
```

```
    print(f"Number of words: {word_count}")
    print(f"Number of characters: {char_count}")
except FileNotFoundError:
    print("Error: 'report.txt' could not be found.")
print("-" * 30)
```

Output:

```
--- File Statistics Counter ---
Analysis of 'report.txt':
Number of lines: 2
Number of words: 20
Number of characters: 104
-----
```

Result:

The Python program successfully computed and displayed the total number of lines, words, and characters (including newlines and spaces) in the file report.txt, fulfilling the objective of computing file statistics.

Experiment 21: Array (List) Manipulations

Aim:

To demonstrate common operations on an array (using a Python list), including creation, display, appending, insertion, and reversal.

Algorithm:

1. **Start** the program.
2. **Create:** Initialize a list named **my_array**.
3. **Display:** Print the original list.
4. **Append:** Add an element to the end of the list using **append()**.
5. **Insert:** Add an element at a specified index using **insert()**.
6. **Reverse:** Reverse the order of elements using **reverse()**.
7. **Display:** Print the list after each modification to show the result.
8. **Stop** the program.

Program:

```
# 21. Write a program to create, display, append, insert and reverse the items in an array.  
print("--- Array (List) Operations ---")
```

```
# 1. Create an array (list)  
my_array = [10, 20, 40, 50]  
print(f"1. Original Array: {my_array}")
```

```
# 2. Append an item  
my_array.append(60)  
print(f"2. After Appending 60: {my_array}")
```

```
# 3. Insert an item at index 2  
my_array.insert(2, 30)  
print(f"3. After Inserting 30 at index 2: {my_array}")
```

```
# 4. Reverse the array  
my_array.reverse()  
print(f"4. After Reversing: {my_array}")  
print("-" * 30)
```

Output:

Result:

The Python program successfully demonstrated the creation and fundamental manipulation techniques on a list, including `append()`, `insert()`, and `reverse()`, validating the array manipulation experiment.

Experiment 22: Matrix Operations

Aim:

To write a Python program to perform addition, transposition, and multiplication on two 2x2 matrices.

Algorithm:

1. **Start** the program.
2. Define two 2x2 matrices, **matrix1** and **matrix2**, as nested lists.
3. Initialize a **result** matrix of the same size with zeros.
4. **Addition:** Iterate through each element of the matrices and store the sum in the result matrix. Print the result.
5. **Transposition:** Iterate through matrix1 and store the element matrix1[j][i] into result[i][j]. Print the result.
6. **Multiplication:** Use three nested loops to calculate the **dot product** of the rows of matrix1 and columns of matrix2. Store the sum in the result matrix. Print the result.
7. **Stop** the program.

Program:

```
# 22. Write a program to add, transpose and multiply two matrices.
```

```
matrix1 = [[1, 2], [3, 4]]
matrix2 = [[5, 6], [7, 8]]
result = [[0, 0], [0, 0]]
```

```
print("--- Matrix Operations ---")
print("Matrix 1:", matrix1)
print("Matrix 2:", matrix2)
print("-" * 25)
```

```
# 1. Matrix Addition
```

```
print("\n--- Addition ---")
for i in range(len(matrix1)):
    for j in range(len(matrix1[0])):
        result[i][j] = matrix1[i][j] + matrix2[i][j]
```

```
for row in result:
    print(row)
```

```
# 2. Matrix Transposition (of Matrix 1)
```

```
print("\n--- Transposition (of Matrix 1) ---")
# Re-initialize result for Transposition
result = [[0, 0], [0, 0]]
```

```

for i in range(len(matrix1)):
    for j in range(len(matrix1[0])):
        result[i][j] = matrix1[j][i]

for row in result:
    print(row)

# 3. Matrix Multiplication
result = [[0, 0], [0, 0]]
print("\n--- Multiplication ---")
for i in range(len(matrix1)):
    for j in range(len(matrix2[0])):
        for k in range(len(matrix2)):
            result[i][j] += matrix1[i][k] * matrix2[k][j]

for row in result:
    print(row)
print("-" * 25)

```

Output:

Result:

The Python program successfully implemented the logic for matrix addition, transposition, and multiplication on two 2x2 matrices using nested lists, verifying the principles of matrix algebra through programming.

Experiment 23: Object-Oriented Shapes

Aim:

To use object-oriented programming to create a Shape parent class and Circle, Square, and Triangle subclasses, each with methods to calculate area and perimeter.

Algorithm:

1. Import the **math** module.
2. Define a base class **Shape** with placeholder `area()` and `perimeter()` methods.
3. Define a Circle class that inherits from Shape.
 - a. Initialize it with a radius.
 - b. Override `area()` and `perimeter()` with circle-specific formulas (πr^2 and $2\pi r$).
4. Define a Square class that inherits from Shape.
 - a. Initialize it with a side length.
 - b. Override `area()` (side^2) and `perimeter()` ($4 \times \text{side}$) methods.
5. Define a Triangle class that inherits from Shape.
 - a. Initialize it with three sides *a*, *b*, *c*.
 - b. Override `perimeter()` to sum the sides ($a+b+c$).
 - c. Override `area()` to use Heron's formula.
6. Create instances (objects) of each shape subclass.
7. Call the `area()` and `perimeter()` methods for each object and print the results.
8. **Stop** the program.

Program:

```
# 23. Create a class Shape with methods for area and perimeter.
```

```
# Implement subclasses for Circle, Triangle, and Square.
```

```
import math
```

```
class Shape:
```

```
    def area(self):
```

```
        pass
```

```
    def perimeter(self):
```

```
        pass
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        self.radius = radius
```

```
    def area(self):
```

```
        return math.pi * (self.radius ** 2)
```

```
    def perimeter(self):
```

```
    return 2 * math.pi * self.radius
```

```
class Square(Shape):
```

```
    def __init__(self, side):
```

```
        self.side = side
```

```
    def area(self):
```

```
        return self.side ** 2
```

```
    def perimeter(self):
```

```
        return 4 * self.side
```

```
class Triangle(Shape):
```

```
    def __init__(self, a, b, c):
```

```
        self.a = a
```

```
        self.b = b
```

```
        self.c = c
```

```
    def perimeter(self):
```

```
        return self.a + self.b + self.c
```

```
    def area(self):
```

```
        s = self.perimeter() / 2 # Semi-perimeter
```

```
        # Heron's formula
```

```
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))
```

```
# --- Demonstration ---
```

```
print("--- Object-Oriented Shapes ---")
```

```
my_circle = Circle(radius=7)
```

```
print(f"Circle (radius=7):")
```

```
print(f" Area: {my_circle.area():.2f}")
```

```
print(f" Perimeter: {my_circle.perimeter():.2f}\n")
```

```
my_square = Square(side=5)
```

```
print(f"Square (side=5):")
```

```
print(f" Area: {my_square.area()}")
```

```
print(f" Perimeter: {my_square.perimeter()}\n")
```

```
my_triangle = Triangle(a=6, b=6, c=6) # Equilateral triangle
```

```
print(f"Triangle (sides=6, 6, 6):")
```

```
print(f" Area: {my_triangle.area():.2f}")
```

```
print(f" Perimeter: {my_triangle.perimeter()}\n")
```

```
print("-" * 30)
```

Output:**Result:**

The experiment successfully implemented the principles of Object-Oriented Programming (OOP) in Python by creating a base Shape class and specialized subclasses (Circle, Square, Triangle) with overridden methods to correctly calculate the area and perimeter of each shape, demonstrating inheritance and polymorphism.
